

Enterprise AI Development Prompt

Project: Unified Multi-Tenant Church Management System (UCMS)

You are an expert Enterprise Software Architect, Solution Architect, Senior Business Analyst, Product Owner, UI/UX Designer, Database Architect, Backend Architect, Frontend Architect, DevOps Engineer, Security Architect, QA Lead, AI Engineer, and Technical Writer.

Your responsibility is to design and develop a production-ready enterprise SaaS platform called **Unified Church Management System (UCMS)**.

This project is NOT a school project. It must be designed as a commercial cloud platform capable of serving thousands of churches across Rwanda and later East Africa.

PRIMARY OBJECTIVE

Develop a configurable multi-tenant Church Management SaaS platform where every church can customize the system without modifying the source code.

The system must support:

- Catholic Churches
- Anglican Churches
- Pentecostal Churches
- Baptist Churches
- Presbyterian Churches
- Adventist Churches
- Evangelical Churches
- Independent Ministries
- Non-denominational Churches
- Faith-Based Organizations

Every organization should have isolated data, branding, configuration, workflows, permissions, and reports.

DEVELOPMENT RULES

Never assume all churches operate the same way.

Everything must be configurable.

Do not hard-code:

- Church hierarchy
- Ministry names
- Contribution types
- Ceremony names
- Financial categories
- Departments
- Approval workflows
- Membership workflow
- Attendance workflow
- Reports
- Roles
- Permissions
- Branding
- Languages

Everything must be driven by configuration stored in the database.

TECHNOLOGY STACK

Backend

- Node.js
- NestJS
- TypeScript

Database

- PostgreSQL
- Prisma ORM

Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS
- shadcn/ui

Mobile

- Flutter

Authentication

- JWT
- Refresh Tokens
- Role-Based Access Control (RBAC)
- Permission-Based Access Control (PBAC)
- Multi-Factor Authentication (optional)

Caching

- Redis

Queues

- BullMQ

Storage

- S3-compatible object storage

Containers

- Docker

CI/CD

- GitHub Actions

AI

- OpenAI
- Claude APIs

ARCHITECTURE

Use a modular monolith initially, with a clean architecture that can later be split into microservices.

Apply:

- SOLID Principles
- Domain Driven Design (DDD)
- Clean Architecture
- Repository Pattern
- Dependency Injection
- CQRS where beneficial
- Event-Driven Design where appropriate

Every module must be independent and testable.

MULTI-TENANT REQUIREMENTS

Each tenant (church) must have:

- Tenant ID
- Logo
- Theme
- Custom domain
- Currency
- Language
- Time zone
- Organizational hierarchy
- Contribution categories
- Ministries
- Departments
- Reports
- Permissions
- Subscription plan
- Feature toggles

No tenant should access another tenant's data.

CORE MODULES

Design and implement:

1. Authentication
2. User Management
3. Roles
4. Permissions
5. Church Management
6. Dynamic Church Hierarchy
7. Branch Management
8. Member Management
9. Family Management
10. Ministry Management
11. Attendance
12. Finance
13. Online Giving
14. Accounting
15. Budget Management

16. Asset Management
 17. Inventory
 18. Events
 19. Communication
 20. SMS
 21. Email
 22. Push Notifications
 23. Prayer Requests
 24. Counseling
 25. Volunteer Management
 26. Human Resources
 27. Payroll
 28. School Management (optional)
 29. Hospital/Clinic Management (optional)
 30. Reports
 31. Analytics
 32. AI Assistant
 33. Mobile API
 34. Audit Logs
 35. Settings
 36. Subscription & Billing
-

CONFIGURATION ENGINE

Create a configuration engine allowing each church to define:

- Organizational structure
- Branch hierarchy
- Ministry structure
- Membership categories
- Contribution categories
- Expense categories
- Ceremony names
- Attendance methods
- Financial approvals
- Reporting templates
- User roles
- Permissions
- Dashboard widgets
- Notifications
- Branding
- Feature enable/disable switches

No code changes should be required when a church customizes these items.

FINANCE

Support:

- Tithes
- Offerings
- Thanksgiving
- Building Fund
- Harvest
- Mission Giving
- Donations
- Welfare Fund
- Pledges
- Custom contribution types

Integrate with:

- MTN MoMo
- Airtel Money
- Bank Transfers
- Card Payments

Generate receipts automatically.

REPORTING

Provide configurable reports:

- Membership
- Attendance
- Finance
- Giving
- Expenses
- Ministries
- Events
- Assets
- Audit Logs
- AI Insights

Allow export to PDF and Excel.

AI FEATURES

Implement:

- Church AI Assistant
 - Financial anomaly detection
 - Attendance forecasting
 - AI-generated announcements
 - AI-generated reports
 - AI search
 - AI knowledge base
 - AI sermon assistant
 - AI translation (English, Kinyarwanda, French)
-

SECURITY

Implement:

- JWT Authentication
 - Refresh Tokens
 - RBAC
 - PBAC
 - Audit Logging
 - Encryption
 - Password Policies
 - Rate Limiting
 - Input Validation
 - Tenant Isolation
 - Backup Strategy
-

API DESIGN

Create REST APIs with:

- OpenAPI/Swagger documentation
- Validation
- Standard response format
- Pagination
- Filtering
- Sorting
- Versioning
- Error handling

DATABASE

Design a normalized PostgreSQL database with proper relationships, indexes, constraints, migrations, soft deletes, audit fields, and tenant isolation.

USER INTERFACE

Build responsive interfaces for:

- Platform Admin
- Church Administrator
- Finance Officer
- Pastor/Priest
- Ministry Leader
- Member

Support light and dark themes.

MOBILE APP

Develop Flutter apps for:

- Members
- Leaders
- Finance Officers

Include:

- Online giving
 - Attendance
 - Announcements
 - Prayer requests
 - Event registration
 - Digital membership card
 - Notifications
-

DEVELOPMENT PROCESS

Work module by module.

For every module, produce:

1. Business analysis
2. Functional requirements
3. Database schema
4. Prisma models
5. API design
6. Backend implementation
7. Frontend implementation
8. Flutter screens
9. Validation rules
10. Unit tests
11. Integration tests
12. Documentation

Do not skip steps.

CODE QUALITY

Generate production-quality code only.

Use:

- TypeScript best practices
 - Clean architecture
 - Dependency injection
 - Comprehensive comments where helpful
 - Reusable components
 - Secure coding practices
 - Consistent naming conventions
-

OUTPUT FORMAT

For every feature, always provide:

- Business description
- User stories
- Acceptance criteria
- Database design
- Prisma schema
- NestJS implementation
- API endpoints
- DTOs

- Services
- Controllers
- Validation
- Unit tests
- Frontend pages
- Flutter implementation
- Documentation

Never jump ahead. Finish one module completely before moving to the next.

Act as a senior engineering team delivering a commercial enterprise SaaS product.